

OOP & Linked Lists

TIP101

Agenda

1 Welcome 🙌

2 OOP in Python

3 Linked Lists

4 **BREAK**

5 Collaborative Problem Solving

6 Wrap Up

10 minutes

15 minutes

20 minutes

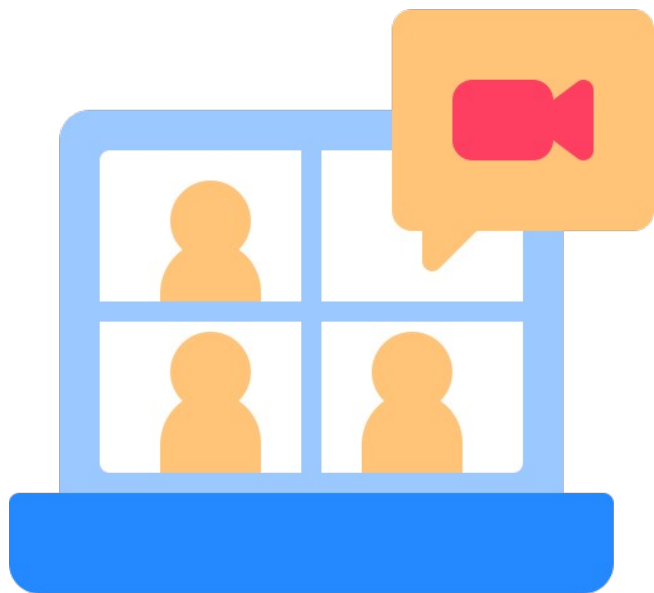
5 minutes

60 minutes

10 minutes

10 minutes

Welcome! 🖐️



**Turn on your
cameras**



**Fill out the entry
ticket**



Icebreaker

5 minutes

Question:

Whether you are a dog person or a cat person (or both!), do you have any pets, and if so how many of what kind?

Answer:

Share your answer in the Zoom chat below. What kinds of pets do you have if any? If you don't have any, share what type you would like to have, if any.

Your Feedback



Glows

- + Students valued Big(O) practice
- + Students learning to plan before implement
- + Explaining reasoning to others



Grows

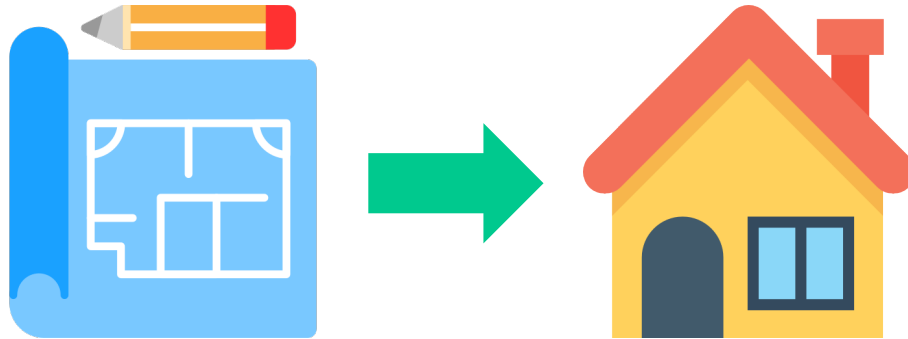
- Spend more time on Big O practice
- Reminder to reach out to us during breakout rooms

15 minutes

OOP in Python

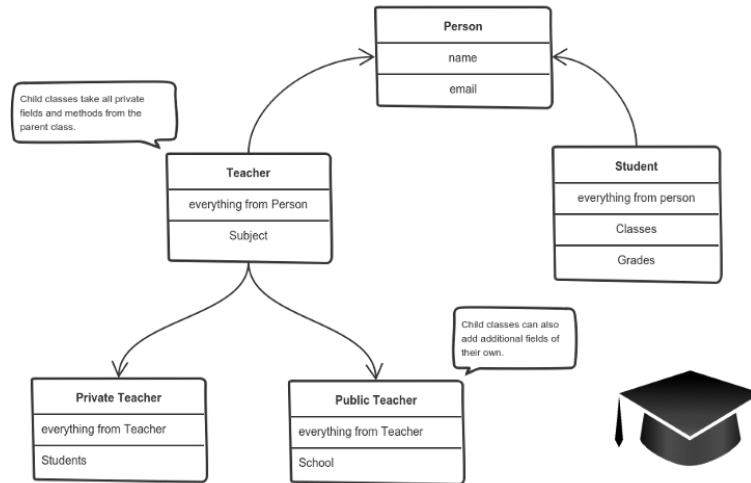
What is a class?

- * A **class** is a blueprint for creating objects.
- * Classes encapsulate data for the object and methods to manipulate that data.



Why do we use classes?

- * Classes are fundamental to object-oriented programming (OOP).
- * They help in modeling real-world entities, making code reusable, and organizing complex programs.



Built-in Python Classes

- * **Strings:** immutable sequences of characters
- * **Lists:** mutable sequences used to store collections of items
- * **Dictionaries:** collections of key-value pairs

Instantiation

- * **Instantiation** is the process of creating an instance (object) of a class.
- * When a class is instantiated, a new object is created with its own set of attributes and methods.

```
my_string = "Hello, World!"
```

```
my_list = [1, 2, 3, 4, 5]
```

```
my_dict = {"key": "value"}
```

Creating a Custom Class

```
class MyClass:  
    def __init__(self, param1, param2):  
        self.param1 = param1  
        self.param2 = param2  
  
    def my_method(self):  
        return self.param1 + self.param2
```

A class is defined using the **class** keyword followed by the class name.

The **__init__** method initializes the object's attributes.

Methods are defined within the class to perform actions on the object's data.

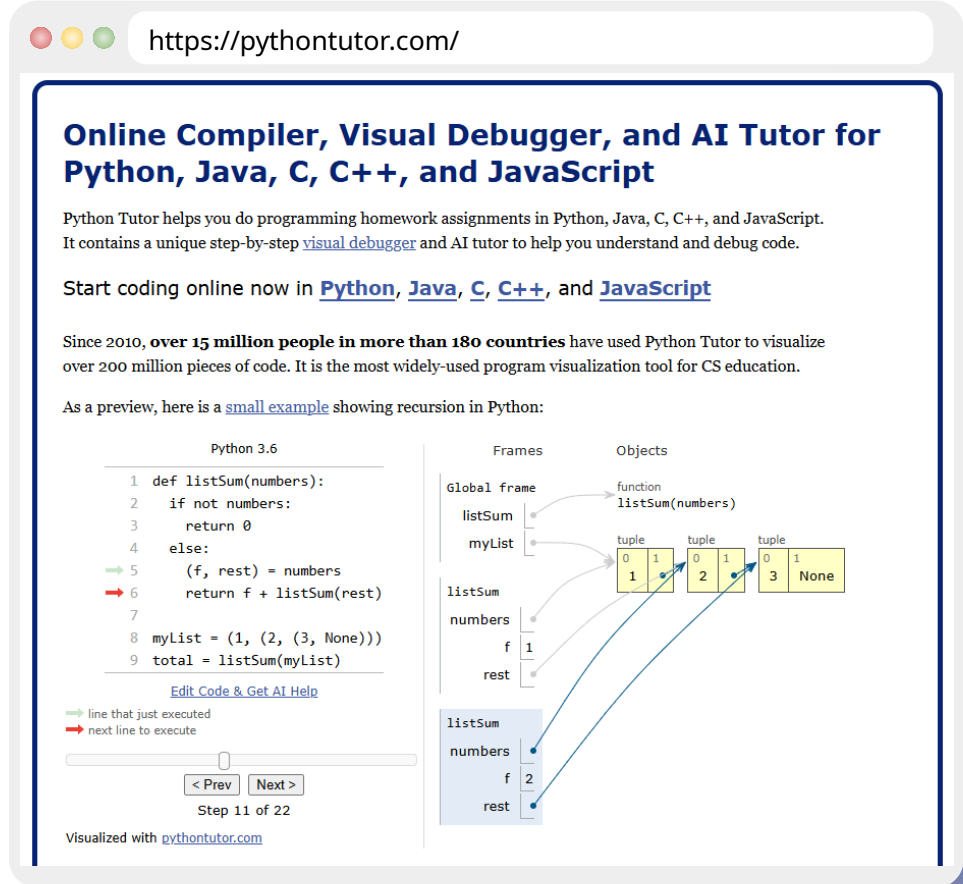
20 minutes

Linked Lists

Python Tutor

You won't be able to use this in an interview, but this is a great visualization of tracing code.

Use it today in your breakout rooms to check against what you think should be occurring!



The screenshot shows the Python Tutor website in a browser window. The URL is <https://pythontutor.com/>. The page title is "Online Compiler, Visual Debugger, and AI Tutor for Python, Java, C, C++, and JavaScript". The text describes the tool's purpose and provides a link to start coding. A preview of a recursion example is shown, including the code for a `listSum` function, the current state of the frames and objects, and navigation controls.

Python 3.6

```
1 def listSum(numbers):
2     if not numbers:
3         return 0
4     else:
5         (f, rest) = numbers
6         return f + listSum(rest)
7
8 myList = (1, (2, (3, None)))
9 total = listSum(myList)
```

[Edit Code & Get AI Help](#)

→ line that just executed
→ next line to execute

Step 11 of 22

Visualized with pythontutor.com

Frames

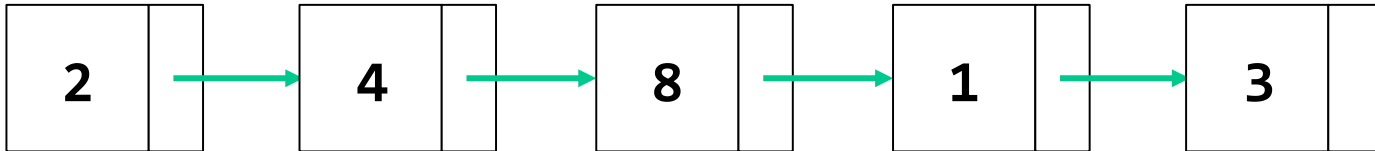
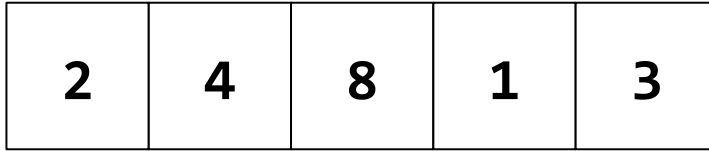
- Global frame
 - listSum → function listSum(numbers)
 - myList → tuple (1, (2, (3, None)))
- listSum
 - numbers → tuple (1, (2, (3, None)))
 - f → 1
 - rest → tuple (2, (3, None))
- listSum
 - numbers → tuple (2, (3, None))
 - f → 2
 - rest → tuple (3, None)

Objects

- function listSum(numbers)
- tuple (1, (2, (3, None)))
- tuple (2, (3, None))
- tuple (3, None)

Arrays vs Linked Lists

Arrays cannot dynamically grow, but linked lists can!



Example

In Python Tutor:

1. Create a Node class.
2. Go through the steps to see what happened.

```
class Node:  
    def __init__(self, value):  
        self.value = value  
        self.next = None  
  
x = Node(2)  
print(x.value)
```


Example

In Python Tutor:

1. Add a Node.
2. Go through the steps again to see what has changed.

```
class Node:  
    def __init__(self, value):  
        self.value = value  
        self.next = None
```

```
x = Node(2)  
y = Node(4)  
print(x.value)
```



Discussion

How can we connect x to y ?



Example

Go through the steps in Python Tutor. What's changed?

```
class Node:  
    def __init__(self, value):  
        self.value = value  
        self.next = None
```

```
x = Node(2)  
y = Node(4)  
x.next = y  
y = None  
print(x.value)
```

Finding a Node

Go through the steps in Python Tutor to trace the code.

```
current = a
while current != None:
    print (current.value)
    current = current.next
```

Break Time!

Take 5 mins to step away from the computer. Feel free to turn off your camera and return promptly after 5 mins.



60 minutes

Collaborative Problem Solving



Breakout Rooms

60 minutes

1. 🏃 Navigate to your breakout room.
2. 🙌 Introduce yourselves!
3. 🧑💻 Work on the problems for your selected unit.

10 minutes

Wrap Up



One Good Thing

Let's reflect on today's session. Maybe you got a question answered, overcame a specific coding challenge, or learned how to do something completely new!

In the chat, share at least one good thing you've learned or experienced during the session today.

Let's celebrate together!



Session Survey Time!





Final Reminders

- ☐ Finish the problem sets for additional practice
- ☐ Take the weekly HackerRank assessment by Sunday @ midnight
- ☐ Visit a **Study Hall Session** to deepen your understanding, work through challenges, and connect with course material alongside your peers!
- ☐ Mock Interview on Thursday! 1 student volunteer interviewing with me!